Complemento Práctico: Lecciones del Ejercicio Integrador

Modelado de un Sistema de Pagos (Posnet)

En el video final de la unidad, desarrollamos un sistema completo que simula el pago con tarjeta de crédito en una terminal (Posnet). Este ejercicio no está diseñado solo para escribir código, sino para entrenar el "músculo de diseño". El verdadero desafío radica en identificar qué objetos necesitamos, cómo se relacionan entre sí y dónde ubicar cada responsabilidad. A lo largo de este documento, destilaremos las conclusiones más importantes del proceso: desde cómo evitar el acoplamiento excesivo hasta cómo aplicar el principio "Tell, Don't Ask" en un escenario real de validación de saldos y cuotas. Cada decisión de diseño que tomamos tiene un fundamento conceptual que nos ayudará a construir sistemas más mantenibles y extensibles.

Contenido

Identificación y Modelado de Entidades

Cómo discernir los objetos clave y definir sus atributos y comportamientos.

Relaciones y Comunicación entre Objetos

Estrategias para que los objetos interactúen con bajo acoplamiento y alta cohesión.

Principios de Diseño Orientado a Objetos

Aplicación de SOLID, encapsulamiento y polimorfismo en la construcción del sistema.

Refactorización y Mejora del Código

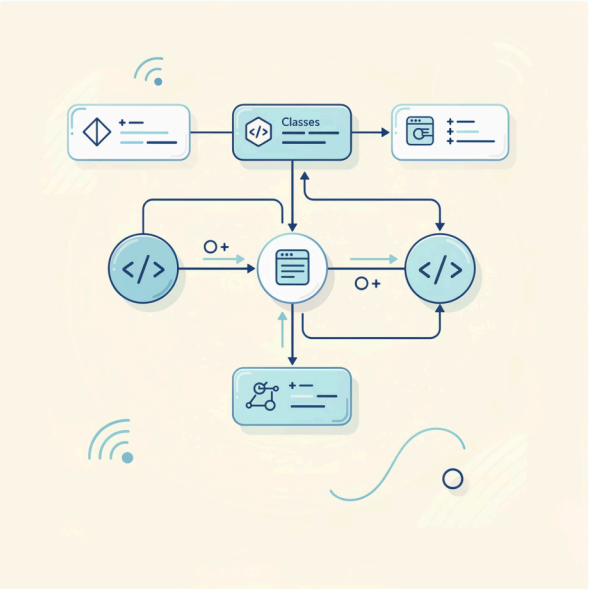
Técnicas para optimizar la estructura del código sin alterar su funcionalidad externa.

Beneficios de un Buen Diseño

Impacto de un diseño robusto en la mantenibilidad, escalabilidad y reusabilidad del sistema.

El Modelo Mental: Identificando Entidades

Abstracción del Problema



Antes de tocar el teclado, el video nos invita a pensar en los actores. El problema plantea: "Una persona va con su tarjeta a un comercio y usa un posnet". Esta simple frase contiene todas las pistas que necesitamos para identificar nuestras entidades principales. El arte del diseño orientado a objetos comienza precisamente aquí: en la capacidad de extraer del lenguaje natural los sustantivos que se convertirán en nuestras clases.

La pregunta fundamental es: ¿qué objetos del mundo real necesitamos representar en nuestro sistema? No se trata de modelar todo lo que existe en un comercio, sino únicamente aquello que tiene comportamiento relevante para resolver nuestro problema específico. Esta capacidad de abstracción selectiva es una de las habilidades más importantes en la programación orientada a objetos.

Las Cuatro Entidades Fundamentales

Posnet

Es el agente activo del sistema. Es quien ejecuta la transacción, valida las condiciones y genera el recibo o comprobante. Tiene la responsabilidad de orquestar todo el proceso de pago.

Tarjeta de Crédito

Es el medio de pago. Contiene datos críticos como el saldo disponible, el banco emisor y el titular. Conoce su propio estado y puede validar operaciones.

Persona (Titular)

Representa al dueño de la tarjeta. Aunque su rol parece secundario, es importante para la integridad del modelo y futuras extensiones del sistema.

Recibo (o Comprobante)

Es el resultado final de la operación. Registra todos los detalles de la transacción: monto, fecha, titular y número de cuotas. Es la evidencia del pago.

Una lección fundamental de diseño emerge aquí: no creamos una clase "Banco" o "Comercio" porque, para este alcance simplificado, no tenían comportamiento relevante. Solo modelamos lo que el sistema necesita hoy. Este principio de parsimonia nos ayuda a mantener el código simple y enfocado en resolver el problema actual, sin anticipar necesidades futuras que tal vez nunca lleguen.

Relaciones Entre Objetos

Conectando los Puntos: Asociación Unidireccional

Una decisión clave del ejercicio fue determinar cómo relacionar la Persona con la Tarjeta. Este tipo de elecciones arquitectónicas tienen consecuencias profundas en la mantenibilidad del código. La decisión que tomamos fue establecer que TarjetaDeCredito tiene un atributo Persona titular. Esto significa que implementamos una asociación unidireccional: la tarjeta sabe de quién es, pero la persona no mantiene una lista de sus tarjetas.

¿Por qué elegimos este diseño? La respuesta está en el principio de simplicidad. Para el alcance actual del sistema, no necesitamos consultar todas las tarjetas de un cliente. Si implementáramos una relación bidireccional, deberíamos sincronizar ambos extremos cada vez que creamos una tarjeta, agregando complejidad innecesaria. Además, complicaría la carga inicial de datos y aumentaría el acoplamiento entre clases.

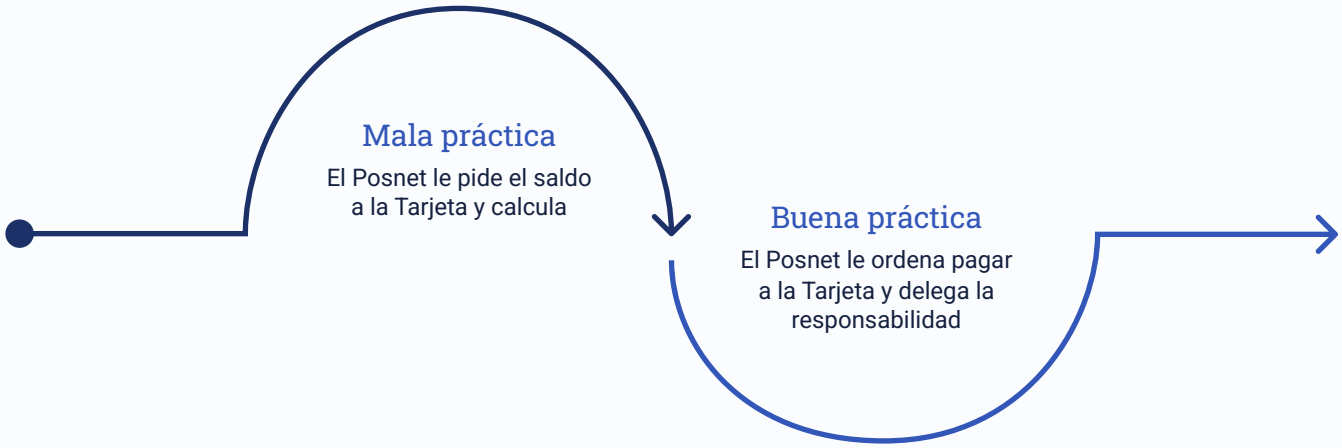
Esta decisión ilustra un principio importante: las relaciones entre objetos no deben reflejar necesariamente todas las conexiones del mundo real, sino únicamente aquellas que el sistema necesita para funcionar correctamente. Si en el futuro necesitáramos consultar las tarjetas de un cliente, podríamos refactorizar hacia una relación bidireccional, pero agregar esa complejidad antes de tiempo sería prematura.

Tell, Don't Ask: El Principio Clave

La Filosofía del Diseño

El punto crítico del video es cuando el Posnet debe realizar el cobro. Aquí aparece uno de los principios más importantes de la programación orientada a objetos: "Tell, Don't Ask" (Decir, no Preguntar). La pregunta fundamental es: ¿quién sabe si hay saldo suficiente? La respuesta correcta es: la Tarjeta.

Este principio nos enseña que no debemos pedirle datos a un objeto para tomar decisiones por él. En lugar de preguntarle al objeto por su estado interno y luego actuar en consecuencia, debemos decirle al objeto qué hacer y dejar que él decida cómo hacerlo basándose en su propio estado. Esta aproximación respeta el encapsulamiento y mantiene la lógica donde corresponde.



Validación de Saldo: Comparación de Diseños

Diseño Incorrecto (Violando el Encapsulamiento)

En este enfoque, el Posnet se inmiscuye en la privacidad de la Tarjeta, preguntando por su saldo y manipulándolo directamente. Este código presenta varios problemas graves:

```
// MAL: El Posnet viola el encapsulamiento
if (tarjeta.getSaldo() >= monto) {
    tarjeta.setSaldo(tarjeta.getSaldo() - monto);
}
```

Este diseño expone el estado interno de la Tarjeta, viola el principio de encapsulamiento y ubica la lógica de negocio en el lugar equivocado. Además, si la Tarjeta tuviera validaciones adicionales (como límite de crédito o fecha de vencimiento), tendríamos que duplicar esa lógica en cada lugar donde se utilice la tarjeta.

Diseño Correcto (Delegación de Responsabilidades)

El Posnet le dice a la Tarjeta: "Intenta pagar esto". La Tarjeta es la que decide si puede hacerlo y ejecuta la acción internamente:

```
// BIEN: Delegación apropiada
if (tarjeta.tieneSaldoDisponible(monto)) {
    tarjeta.descontar(monto);
}
```

En el video, esto se encapsuló aún más elegantemente en un único método `tarjeta.pagar(monto)` que devuelve el Ticket si la operación tuvo éxito o `null` si falló. Este diseño centraliza toda la lógica de validación y descuento en la clase que realmente conoce esa información, haciendo el código más robusto y mantenible.

El Algoritmo de Cuotas

Cálculos Matemáticos en el Lugar Correcto

El ejercicio requería calcular un recargo del 3% por cada cuota si el pago era en plazos. Por ejemplo, un pago en 6 cuotas implica un 18% de recargo sobre el monto base. Esta simple regla de negocio nos enfrenta a una pregunta de diseño fundamental: ¿dónde ubicamos esta lógica? La respuesta correcta determina la calidad arquitectónica de nuestra solución.

1

En el Main

Esta es la peor opción posible. El código quedaría disperso y no reutilizable. Cada vez que necesitemos calcular cuotas, tendríamos que repetir la lógica.

2

En la Tarjeta

Aunque la Tarjeta maneja el saldo, no es quien define las políticas de recargo. Los recargos son decisiones comerciales, no características de la tarjeta.

3

En el Posnet

Esta es la ubicación correcta. El dispositivo Posnet es quien aplica las reglas del comercio, incluidos los recargos por financiación.

Implementación Elegida

El cálculo quedó encapsulado en el método del Posnet que procesa el pago:

```
double montoFinal = montoBase +
    (montoBase * cantCuotas * 0.03);
```

Esta línea de código es simple pero poderosa. Calcula el monto total aplicando el 3% de recargo por cada cuota. Si necesitamos cambiar la política de recargos en el futuro, solo modificamos este método del Posnet sin afectar otras clases.

Detectando el "Code Smell"

El Problema del Acoplamiento y la Ruptura del Encapsulamiento

Como mencionamos, el diseño no nació perfecto. En el video, partimos de una idea intuitiva pero "sucia" y la fuimos puliendo a través de un proceso iterativo de refactorización. La primera etapa crucial en este viaje fue identificar los "olores a mal código" (code smells) que indicaban un diseño deficiente.

Al principio, se observó una fuerte tentación de que el objeto `Posnet` asumiera demasiadas responsabilidades sobre la gestión de la `Tarjeta`. Esto resultaba en un alto acoplamiento, donde el `Posnet` "sabía demasiado" sobre los detalles internos de la `Tarjeta`, rompiendo el principio fundamental del encapsulamiento.

La versión inicial, aunque funcional, ilustraba una mentalidad estructurada donde el `Posnet` actuaba como un "micro-manager", pidiendo el saldo a la tarjeta, calculando la resta y luego guardando el nuevo saldo de forma externa:

```
// MAL DISEÑO (Lo que evitamos)
// El Posnet actúa como un "micro-manager"
double saldoActual = tarjeta.getSaldo();
double montoAPagar = 1000;

if (saldoActual >= montoAPagar) {
    double nuevoSaldo = saldoActual - montoAPagar;
    tarjeta.setSaldo(nuevoSaldo); // ¡Peligro! Modificación externa
}
```

Este enfoque revela una serie de deficiencias. El `Posnet` no debería necesitar saber cómo la `Tarjeta` maneja su saldo internamente. Si la lógica de descuento o validación del saldo cambia en la `Tarjeta` (por ejemplo, si se añaden límites de crédito o comisiones), el `Posnet` también tendría que ser modificado, generando código frágil y difícil de mantener.

> **El Principio "Tell, Don't Ask":** Este "code smell" es una clara violación del principio "Tell, Don't Ask" (Dile, no preguntes). En lugar de preguntar a un objeto por su estado (ej. `getSaldo()`) para tomar una decisión externa (ej. si descontar), se debe pedir al objeto que realice la acción completa (ej. `tarjeta.pagar(monto)`), dejando que sea él quien decida si puede hacerlo y cómo.

Aplicando la Refactorización ("Tell, Don't Ask")

Para corregir el "code smell" de alto acoplamiento y la ruptura del encapsulamiento que habíamos identificado, aplicamos un principio fundamental de la programación orientada a objetos: la delegación, siguiendo la máxima de "Tell, Don't Ask" (Dile, no preguntes). En lugar de que el objeto `Posnet` "preguntara" a la `Tarjeta` por su saldo y luego gestionara la lógica de descuento externamente, le dimos la responsabilidad directa a la clase experta: la `TarjetaDeCredito` misma. Este enfoque permite que cada objeto gestione su propia lógica interna, promoviendo un diseño más robusto y mantenible.

01

Paso 1: Encapsular la lógica del saldo

Creamos el método `tieneSaldoDisponible(monto)` dentro de la clase `Tarjeta`. Ahora, el `Posnet` no necesita saber el monto exacto del saldo; solo necesita una respuesta booleana (`true` o `false`) sobre la viabilidad de la operación.

02

Paso 2: Encapsular el descuento

De manera similar, el método `descontar(monto)` se implementó en la `Tarjeta`. Esto significa que la `Tarjeta` es quien decide cómo se aplica el descuento, encapsulando completamente esta lógica de negocio.

Observen la diferencia en la versión refactorizada, que refleja una verdadera mentalidad orientada a objetos:

```
// BUEN DISEÑO (Aplicado en el video)
// El Posnet delega la responsabilidad
if (tarjeta.tieneSaldoDisponible(montoFinal)) {
    tarjeta.descontar(montoFinal);
    // Generar ticket...
}
```

En esta versión, el `Posnet` no tiene conocimiento de cuánto saldo le queda a la persona ni cómo se realiza el descuento; su única preocupación es si la operación fue exitosa o no. Este diseño mejora significativamente la seguridad del sistema y reduce la complejidad en la clase `Posnet`, haciendo que cada componente sea más autónomo y fácil de entender.

A- Beneficios del Diseño Final

Después de aplicar los principios de "Tell, Don't Ask" y encapsular la lógica en los objetos correctos, nuestro sistema no solo funciona correctamente, sino que también presenta una arquitectura mucho más elegante y fácil de entender. Este diseño final ofrece ventajas sustanciales en términos de legibilidad, mantenibilidad y escalabilidad. A continuación, analizaremos la arquitectura resultante, centrándonos en cómo la refactorización ha impactado la claridad del código, especialmente en el método principal.

La Orquestación del Método Main

Uno de los indicadores más claros de un buen diseño orientado a objetos es la simplicidad del método `main`. En nuestra versión refactorizada, el `main` ha evolucionado de ser un centro de cálculos y decisiones intrincadas a convertirse en un orquestador eficiente, cuyo único propósito es preparar el escenario, iniciar las acciones principales y presentar el resultado. Ya no contiene lógica de negocio compleja ni realiza manipulaciones directas de datos internos de otros objetos.

```
// MAIN LIMPIO Y ORQUESTADOR
public static void main(String[] args) {
    // 1. Preparamos el escenario (Objetos)
    Posnet posnet = new Posnet();
    Persona p = new Persona("40123456", "Juan", "Perez", "111222333",
"juan@mail.com");
    TarjetaDeCredito t = new TarjetaDeCredito("Banco Fake", "1234567890123456",
15000, p, EntidadFinanciera.VISA);

    // 2. Ejecutamos la acción principal
    Ticket ticketGenerado = posnet.efectuarPago(t, 10000, 5);

    // 3. Mostramos el resultado
    System.out.println(ticketGenerado);
}
```

B- Beneficios del Diseño Final

Este enfoque permite que cualquier desarrollador, incluso sin conocer los detalles de implementación de `Posnet` o `TarjetaDeCredito`, comprenda la secuencia general de operaciones casi de inmediato. Los objetos interactúan de manera predecible, delegando responsabilidades de forma clara.

1

Legibilidad Mejorada

El código del `main` se vuelve casi autoexplicativo, sirviendo como una descripción de alto nivel del flujo principal del programa.

2

Separación de Responsabilidades

Cada objeto es responsable de su propia lógica, promoviendo un bajo acoplamiento y una alta cohesión, haciendo el sistema más robusto.

3

Facilidad de Pruebas

Al estar la lógica de negocio encapsulada, es más sencillo escribir pruebas unitarias para cada componente sin depender del contexto completo.

4

Mantenimiento Simplificado

Si la lógica para "efectuar pago" cambia, solo necesitamos modificar la clase `Posnet`, sin afectar el `main` u otras partes del sistema.

Beneficios Clave del Diseño Refactorizado

La aplicación rigurosa de principios de diseño orientado a objetos como "Tell, Don't Ask" y la encapsulación de la lógica de negocio en las entidades correctas, no solo simplificaron la interacción del sistema, sino que también cimentaron una arquitectura con beneficios fundamentales. Dos de los pilares más importantes de este diseño son la responsabilidad clara y cohesiva de cada objeto y la inherentemente mayor extensibilidad del sistema, preparándolo para futuros cambios y adiciones.

Responsabilidad Clara (Cohesión)

En un buen diseño, cada clase debería tener un "trabajo" único y bien definido. Esta claridad asegura que cada componente del sistema sea autocontenido, fácil de entender y de modificar sin afectar a otros. En el ejemplo del video, esta distribución se hizo evidente en la lógica del recargo y la gestión del saldo:

El `Posnet`: Orquestador del Cálculo Comercial

Su responsabilidad se centra en el contexto de la venta: **determinar el precio final de la operación**, incluyendo cualquier recargo asociado a las cuotas. La fórmula `montoFinal = montoBase + (montoBase * cantCuotas * 0.03)` reside aquí porque es el comercio (representado por el `Posnet`) quien define y aplica el costo adicional por financiación, no la tarjeta.

La `Tarjeta`: Custodia de sus Fondos

La `TarjetaDeCredito` asume la responsabilidad exclusiva de **gestionar su propio saldo y sus límites**. Es la única entidad que decide si tiene fondos disponibles para una operación y cómo se realiza el descuento, manteniendo la integridad de su estado interno.

El `Ticket`: Registro Inmutable

El `Ticket` es un reporte inmutable de lo que sucedió en la transacción. Su única función es **almacenar y presentar la información** de un pago ya efectuado, sin lógicas activas de cálculo o validación.

Sistema Extensible

Uno de los mayores logros de este diseño es su capacidad para crecer y adaptarse. Gracias a que el método `efectuarPago` del `Posnet` devuelve un objeto `Ticket` (en lugar de simplemente imprimir en consola o devolver `void`), el sistema está intrínsecamente listo para nuevas funcionalidades sin requerir cambios invasivos en las clases existentes.

Por ejemplo, si mañana surgiera la necesidad de enviar un recibo por correo electrónico, simplemente se tomaría el objeto `Ticket` *retornado y se pasaría a un nuevo "EnviadorDeEmail". Esto demuestra cómo el bajo acoplamiento y la alta cohesión se traducen directamente en un sistema más maleable, donde las nuevas características pueden añadirse como módulos adicionales, sin perturbar el corazón de la aplicación.*

Conclusión: El Arte del Diseño Orientado a Objetos

Más allá del Código

El viaje a través del ejercicio del Posnet nos ofrece una perspectiva valiosa que va más allá de la mera implementación de funcionalidades. Nos enseña que la madurez del programador se revela no solo en lograr que el código "funcione", sino en la habilidad de **ubicar las cosas en su lugar**, distribuyendo responsabilidades de manera inteligente y coherente. Lejos de las prácticas iniciales de concentrar toda la lógica en el método principal o de usar variables globales, la Programación Orientada a Objetos (POO) nos invita a pensar en una arquitectura limpia y modular.

Las Lecciones Esenciales

01

No Temas al Cambio

El código no es una losa de mármol tallada, sino **arcilla** que se moldea. La capacidad de borrar y reescribir métodos enteros, de refactorizar sin miedo, es un signo de un proceso de diseño saludable y adaptable. Esta flexibilidad es clave para la evolución de cualquier sistema.

02

La Comunicación Respetuosa entre Objetos

El verdadero éxito del sistema Posnet no residió en la complejidad de sus cálculos, sino en la elegancia con la que **Posnet conversaba con la Tarjeta** sin invadir su estado interno. Este principio de "Tell, Don't Ask" es fundamental para un bajo acoplamiento y una alta cohesión.

Reflexión Final

Un buen diseño orientado a objetos es aquel que, una vez finalizado, parece **"obvio"**. Si al revisar el código final del Posnet, encuentras natural que la tarjeta de crédito se valide a sí misma y que el Posnet se encargue de calcular los intereses, entonces has capturado la esencia de la programación orientada a objetos: la intuición del diseño emerge cuando cada parte del sistema asume su rol de manera lógica y autónoma.